**Student Name:** Shiva Sai Krishna S **Register Number:** 1911260318

**Course:** CSA0421 - Operating System **GitHub Repo:** krishshiva721/CSA04--Assignment1

# TASK 7: PROCESS CONTROL EVALUATION

### Shell Process Management, Buffer Handling, and Process Lifecycle

## 1. COMPLETED C CODE EXECUTION (TASK 7 IMPLEMENTATION)

```
#define _POSIX_C_SOURCE 200809L #include <stdio.h> #include <stdlib.h> #include <unistd.h>
#include <sys/wait.h> int main(void) { int status; printf("shell: running job 1\n");
fflush(stdout); /* Empty user-space buffer before fork() */ /* TODO 1: Process Creation and
Management */ pid_t pid = fork(); if (pid == 0) { execlp("expr", "expr", "6", "*", "7",
(char*)NULL); _exit(127); /* If exec fails */ } waitpid(pid, &status, 0); printf("job 1 exit status
%d\n", WEXITSTATUS(status)); printf("shell: running job 2\n"); fflush(stdout); pid = fork(); if
(pid == 0) { _exit(3); } waitpid(pid, &status, 0); printf("job 2 exit status %d\n",
WEXITSTATUS(status)); return 0; }
```

## 2. ANALYSIS OF OUTPUT REDIRECTION & BUFFER BEHAVIOR

**Why Flush stdout Before Fork:** Standard output is line-buffered when writing to an interactive terminal, but becomes fully block-buffered when redirected to a file. Without `fflush(stdout)`, string bytes remain inside user-space C stdio memory buffers. When `fork()` duplicates the process address space, child processes inherit identical unflushed buffer copies. Upon standard program termination, both parent and child flush their inherited buffers into the file, causing duplicate output strings (e.g., *"shell: running job 2"* appearing twice).

**Zombies vs. Orphans:** A *Zombie* is a terminated process whose parent has not yet invoked `waitpid()` to collect its termination exit status (retaining PCB entry). An *Orphan* process occurs when a parent terminates before its child; the OS `init / systemd` process (PID 1) adopts the orphan and automatically reaps its exit state.

## 3. ALTERNATIVE CONCEPT & COMPLEXITY ANALYSIS

> **Alternative Scenario: Process Creation (fork + exec) vs. POSIX Spawn (posix_spawn) vs. Threads**
>
> - **fork() + exec() Overhead:** Clones virtual page table structures using Copy-on-Write (COW). For a server consuming 16 GB memory, page table copying introduces noticeable latency (`O(M)` page entry allocation time).
> - **posix_spawn() Performance:** Avoids duplicating complete page table hierarchies by utilizing `vfork()` semantics safely under the hood. Execution latency drops by ~60% when spawning 1,000 commands concurrently.
> - **Thread Creation vs. Process Creation:** Creating threads requires only allocating a new user stack frame and thread control block, operating entirely within shared memory address space (`Space = O(1)` incremental overhead, `Time < 5 µs` creation latency).